CSC3002F 2026
OPERATING SYSTEMS II
SCHEDULING ASSIGNMENT

LECTURER: MICHELLE KUTTEL

ALLEGRA THE BARMAN: COMPARING SCHEDULING POLICIES FOR BARTENDING

Version 2 Tuesday, 5 May 2026
(changes in yellow)

Figure 1 produced with Gemini 3 w Nano Banana

INTRODUCTION

The aim of this assignment is to increase your understanding of **CPU process scheduling,** particularly the different **metrics** by which schedulers are compared, and to give you some experience evaluating and comparing basic scheduling algorithms using a **simulation**.

> Computer simulation is used to model the behaviour of a complex system; the theory is that the simplified model has *predictive properties* and can tell us something about a real world system.

You are given a Java package for simulating a version of process scheduling. This is a simple simulation (the output is just text) involves a scenario of a busy bar where patrons arrive at random times throughout the evening and place orders for one to eight drinks each. The drink orders are filled by **Allegra the Barman** (who here is the CPU and scheduler in one). In this analogy, each **patron** is a **process**, each of their **drink orders** is a CPU burst, each **patron drinking** is an I/O wait. This has been carefully coded so that you can have reproducible workloads for testing (numbers of patrons and numbers of drinks ordered).

For this assignment, Allegra the Barman uses one of several possible scheduling algorithms. *Note that none of these algorithms use pre-emption at the level of an individual drink order*, as barmen tend to mess this up and it really annoys patrons. So once Allegra the Barman has started with a drink, she finishes it before moving onto the next one.

ASSIGNMENT SPECIFICATIONS

Your task is to use the simple Java package provided to compare experimentally the performance of the

- **First-Come First-Served** (FCFS);
- **Shortest Job First** (SJF)
- **Priority** and
- **Multilevel Feedback Queue** (MLFQ) scheduling policies.

schedules for **Allegra the Barman** with respect to the metrics **response time**, **waiting time** (both per order, and the total for a patron) and **turnaround time,** as well as **throughput, predictability**, **fairness** and **possibility of starvation**. (I am happy to clarify what these

mean for Allegra the Barman in class or on the MS Team, if asked well ahead of the deadline)

You must then make, and justify, a **recommendation** for the **best** scheduling algorithm for Allegra the Barman, taking into consideration **which metric(s) would be the most important for a bar owner**.

1. Performance Comparison

To compare the performance you will need to write various times to files for analysis: you will need to add code to do this. You should also experiment with **shell scripts** to automate the testing for different scenarios.

Note that you will have to decide how best to test the algorithms, bearing in mind that **experiments** must be run **multiple times** and with a **range of input sizes** for reliable data.

You must also ensure that you compare scheduling algorithms fairly, using the **same workloads**.

2. Report

You must submit a **short report** containing the following sections.

a. Methods

Explain clearly what you did to generate data, in enough detail that someone else could repeat your experiments.
You must also explain clearly what you did to **validate the code** and **the data** generated.

b. Results

Show and **explain graphs** of the behaviours of all scheduling algorithms across **all metrics.** You must compare and contrast the average values, the median values and the distribution of each metric, for each algorithm. You should think carefully about how to design your graphs so that the results are clear.

You need to comment on whether the algorithms **behave as expected**: was there any result that surprised you? Then you need to discuss which algorithm performs **best for each metric** and why this is. You must then consider the **predictability**, **fairness** and **possibility of starvation** in this scenario for all algorithms.

You should spend some time thinking about what you should graph and how best to show the data.

c. Conclusions

Draw conclusions on the basis of the data you have presented, recommending a best scheduling algorithm for Allegra the Barman and justifying your recommendation.

d. Statement on AI usage

You may use generative AI to assist with coding, but what and how you used it should be briefly acknowledged in your report. AI may be used for limited coding assistance.

However, AI may not be used for fraud: e.g. to fabricate experiments, generate fake outputs, invent graphs, or write results/conclusions not supported by actual runs.

3. Optional Bonus Extension

Implement your own improved scheduling algorithm for Allegra the Barman, and compare its performance to the other four. For this, your will need to explain the algorithm in the Methods, and report on it in the results and conclusions. [NOTE: the maximum bonus mark for this is +10%; with a cap at 100%, it's really for fun.]

Constraints

- You have to work from / profile / extend the code provided; you may not rewrite it from scratch or change the fundamental way it works.
- You may not change the `DrinkOrder.java` or the `Patron.java` classes at all.
- You may ~~only~~ **not** change the `Barman.java` class **except** for the `recordCompletedOrder()` method. [If you are implementing the bonus aspect, then you can change other methods, of course.]
- You may not change the command-line arguments for the code.
- You may not change the arrival times for the patrons.

Submission

- Submit the **code** to the automarker as a zipped **archive.** Your submission archive must contain the following.
    - **All the files** needed to run your solution, including any shell scripts.
    - A **Makefile** so that the following command **must run your solution** correctly.
      ```
      make run ARGS="30 2 5 30"
      ```
    - **a GIT usage log (**as a .txt file, use `git log –all` to display all commits and save).
    - All the data files used to generate your graphs, in a subfolder called **results**.
- Submit a separate document in **PDF format** to the assignment containing your report. It should be about 4-10 pages in length, and report on all the aspects listed above.

NOTE: you must submit your assignment as per the instructions above. There are no appeals on this if you do not do so, so check and double-check that you have done this correctly.